

Tutorial: Clonando o Jogo CONEXO com React

Desafio da Aula — inspirado no tutorial oficial do React (Tic-Tac-Toe)

Referência: <https://react.dev/learn/tutorial-tic-tac-toe>

Termo

CONEXO é um jogo de palavras onde o jogador deve encontrar grupos de 4 palavras que compartilham uma conexão temática. A cada rodada há 16 palavras e 4 grupos secretos. O objetivo é identificar todos os grupos em até 4 erros.

Contexto

Item	Detalhe
Jogo original	NYT Connections (adaptação brasileira: Conexo)
Tecnologia	React (Vite ou Create React App)
Conceitos aplicados	<code>useState</code> , componentes, props, listas, eventos
Nível	Iniciante–Intermediário
Tempo estimado	2–3 horas

Por que React para esse jogo?

O Conexo tem **estado complexo**: quais palavras estão selecionadas, quais grupos foram resolvidos, contagem de erros, fim de jogo. O React gerencia esse estado de forma declarativa — a UI sempre reflete o estado atual, sem manipulação manual do DOM.

Conexo Dialed — Arquitetura do Projeto

Estrutura de pastas

```
conexo/
├─ src/
│   ├─ App.jsx           ← componente raiz
│   └─ data/
│       └─ puzzle.js     ← dados do puzzle (grupos e palavras)
│       └─ components/
│           ├─ WordTile.jsx ← botão de palavra individual
│           └─ SolvedBanner.jsx ← banner de grupo resolvido
├─ index.html
└─ package.json
```

Fluxo de dados (prop drilling simples)

```
App (estado central)
├─ SolvedBanner[] ← recebe: group
└─ WordTile[]     ← recebe: word, selected, onClick
```

Passo a Passo

1. Definir os dados do puzzle

Crie `src/data/puzzle.js`:

```
export const PUZZLE = {
  groups: [
    {
      category: "🌊 Elementos da Natureza",
      color: "#22d3ee",
      bg: "#083344",
      words: ["CHUVA", "VENTO", "NEVE", "RELÂMPAGO"],
    },
    {
      category: "🎸 Gêneros Musicais",
      color: "#a78bfa",
      bg: "#2e1065",
      words: ["ROCK", "SAMBA", "JAZZ", "BLUES"],
    },
    {
      category: "🐾 Animais da Selva",
      color: "#4ade80",
      bg: "#052e16",
      words: ["ONÇA", "MACACO", "TUCANO", "CAPIVARA"],
    }
  ]
}
```

```

    },
    {
      category: "🍕 Comidas Italianas",
      color: "#fb923c",
      bg: "#431407",
      words: ["PIZZA", "RISOTO", "LASANHA", "GNOCCHI"],
    },
  ],
};

```

💡 **Conceito:** separar os dados da lógica facilita trocar o puzzle sem tocar no código do jogo.

3. Criar o componente `WordTile`

```

// src/components/WordTile.jsx
export default function WordTile({ word, selected, solved, solvedGroup, onClick }) {
  const style = solved
    ? { background: solvedGroup.bg, borderColor: solvedGroup.color, color: solved
      : selected
    ? { background: "#1e293b", borderColor: "#f8fafc", color: "#f8fafc" }
    : {} };

  return (
    <button onClick={onClick} disabled={solved} style={style} className="tile">
      {word}
    </button>
  );
}

```

💡 **Conceito:** esse componente é **puro** — não tem estado próprio. Ele apenas exibe o que recebe via props. Isso o torna fácil de testar e reutilizar.

4. Criar o componente `SolvedBanner`

```

// src/components/SolvedBanner.jsx
export default function SolvedBanner({ group }) {
  return (
    <div style={{ background: group.bg, borderColor: group.color, color: group.co
      className="solved-banner">
      <strong>{group.category}</strong>
    </div>
  );
}

```

```
        <span>{group.words.join(" · ")}</span>
      </div>
    );
  }
}
```

5. Estado central em `App.jsx`

```
import { useState } from "react";
import { PUZZLE } from "../data/puzzle";
import WordTile from "../components/WordTile";
import SolvedBanner from "../components/SolvedBanner";

// Pega todas as palavras em array plano e embaralha
const ALL_WORDS = PUZZLE.groups.flatMap(g => g.words);

function shuffle(arr) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

export default function App() {
  const [tiles, setTiles] = useState(() => shuffle(ALL_WORDS));
  const [selected, setSelected] = useState([]); // palavras selecionadas
  const [solvedGroups, setSolvedGroups] = useState([]); // grupos resolvidos
  const [mistakes, setMistakes] = useState(0);
  const [message, setMessage] = useState("");

  // ... lógica abaixo
}
```

 **Conceito:** `useState` com função inicializadora (`() => shuffle(...)`) roda o embaralhamento **uma única vez**, na montagem do componente.

6. Lógica de seleção

```
function toggleSelect(word) {
  setSelected(prev =>
```

```

    prev.includes(word)
      ? prev.filter(w => w !== word)          // deseleciona
      : prev.length < 4 ? [...prev, word] : prev // seleciona (máx 4)
  );
}

```

7. Lógica de envio (validação do grupo)

```

function handleSubmit() {
  if (selected.length !== 4) return;

  const group = PUZZLE.groups.find(g =>
    selected.every(w => g.words.includes(w))
  );

  if (group) {
    setSolvedGroups(prev => [...prev, group]);
    setSelected([]);
    setMessage(`✅ ${group.category} – Correto!`);
  } else {
    setMistakes(prev => prev + 1);
    setMessage("❌ Tente novamente");
  }
}

```



Conceito: `Array.every()` verifica se **todas** as palavras selecionadas pertencem ao mesmo grupo.

8. Renderizar o JSX

```

return (
  <div className="app">
    <h1>CONEXO</h1>

    {/* Grupos já resolvidos */}
    {solvedGroups.map(g => <SolvedBanner key={g.category} group={g} />)}

    {/* Grade de palavras (apenas não resolvidas) */}
    <div className="grid">
      {tiles
        .filter(w => !solvedGroups.flatMap(g => g.words).includes(w))
      }
    </div>
  </div>
)

```

```

        .map(word => (
            <WordTile
                key={word}
                word={word}
                selected={selected.includes(word)}
                onClick={() => toggleSelect(word)}
            />
        ))
    </div>

    <p>{message}</p>
    <p>Erros: {mistakes} / 4</p>

    <button onClick={handleSubmit} disabled={selected.length !== 4}>
        Enviar
    </button>
    <button onClick={() => setTiles(t => shuffle([...t]))}>
        Embaralhar
    </button>
</div>
);

```

Checklist de funcionalidades

- Exibir 16 palavras embaralhadas
 - Selecionar/deselecionar palavras (máx 4)
 - Validar se as 4 palavras formam um grupo
 - Exibir grupos resolvidos com cor
 - Contar erros (máx 4)
 - Mensagem de vitória / derrota
 - Botão de embaralhar
 - Botão de reiniciar
-

Desafios Extras

Nível	Desafio
-------	---------

★	Adicionar animação de shake quando errar
★★	Mostrar "quase lá!" quando 3 de 4 palavras estão certas
★★★	Buscar um puzzle aleatório de uma API externa
★★★★	Criar modo multiplayer com socket.io

Referências

- [Tutorial oficial React — Tic-Tac-Toe](#)
 - [NYT Connections \(jogo original\)](#)
 - [Documentação useState](#)
 - [Array.flatMap MDN](#)
-

Tutorial criado como desafio da aula — clone de web game com React.